



COMPSCI 130

Introduction to Software Fundamentals

Queues

Agenda & Reading

► Agenda

- What is a Queue?
- Queue Abstract Data Type (ADT)
- Implementing a queue using a list
- Using the Queue ADT to solve problems
- A Circular Queue

► Reading

- [Online Textbook](#), Chapter 14



Can you think of other examples of queues?



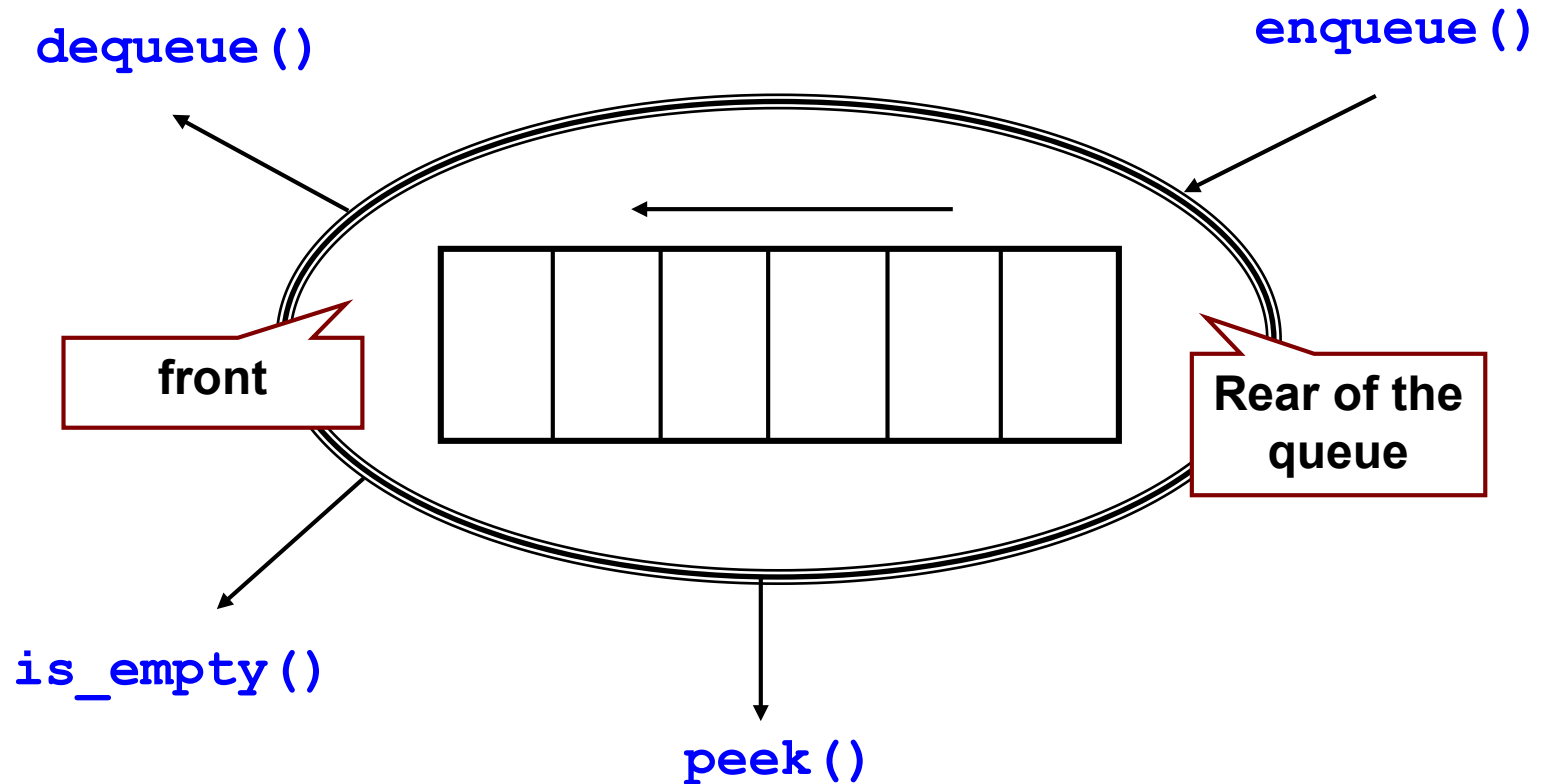


What is a Queue?

- ▶ Queues are appropriate for many real-world situations
 - ▶ Example: A line to buy a movie ticket
 - ▶ Computer applications, e.g. a request to print a document
- ▶ A queue is an **ordered collection** of items where the addition of **new** items happens at one end (the rear or back of the queue) and the **removal** of existing items always takes place at the other end (the front of the queue).
 - ▶ New items enter at the **back**, or rear, of the queue
 - ▶ Items leave from the **front** of the queue
 - ▶ **First-in, first-out (FIFO)** property:
 - ▶ The first item inserted into a queue is the first item to leave

More formally...

Queues implement the FIFO (first-in first-out) policy:
e.g. the printer / job queue!





ADT Queue Operations

- ▶ What are the operations which can be used with a Queue Abstract Data?
 - ▶ Create an empty queue:
 - ▶ Determine whether a queue is empty:
 - ▶ Add a new item to the queue:
 - ▶ **enqueue**
 - ▶ Remove from the queue the item that was added earliest:
 - ▶ **dequeue**
 - ▶ Retrieve from the queue the item that was added earliest:
 - ▶ **peek**



The Queue Abstract Data Type

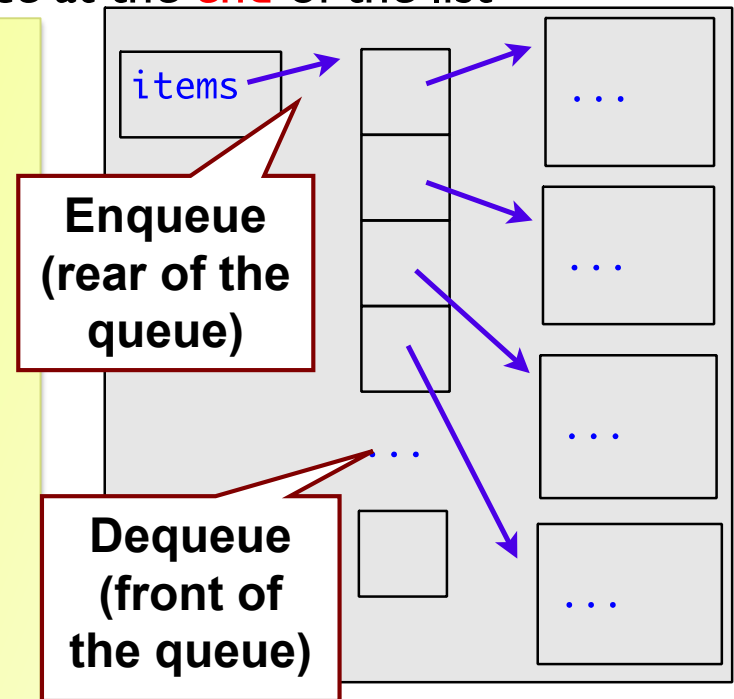
- ▶ Queue() creates a new queue that is empty.
 - ▶ It needs no parameters and returns an empty queue.
- ▶ enqueue(item) adds a new item to the **rear** of the queue.
 - ▶ It needs the item and returns nothing.
- ▶ dequeue() removes the **front** item from the queue.
 - ▶ It needs no parameters and returns the item. The queue is modified.
- ▶ peek() returns the item from the front of the queue.
 - ▶ It needs no parameters and returns the item. The queue is not modified.
- ▶ is_empty() tests to see whether the queue is empty.
 - ▶ It needs no parameters and returns a boolean value.
- ▶ size() returns the number of items in the queue.
 - ▶ It needs no parameters and returns an integer.

Queue Implementation in Python

- ▶ We use a python **List** data structure to implement the queue.
- ▶ Remember:
 - ▶ the **addition** of new items takes place at the **beginning** of the list
 - ▶ the **removal** of existing items takes place at the **end** of the list

```
class Queue:
```

```
    def __init__(self):  
        self.items = []  
    def is_empty(self):  
        return self.items == []  
    def size(self):  
        return len(self.items)  
  
    def enqueue(self, item):  
        self.items.insert(0,item)  
    def dequeue(self):  
        return self.items.pop()
```





Code Example – Result

- ▶ Queue operations, state of the queue, values returned

q = Queue()

```
q.is_empty()
q.enqueue(4)
q.enqueue('dog')
q.enqueue(True)
q.size()
q.is_empty()
q.enqueue(8.4)
q.dequeue()
q.dequeue()
q.size()
```

```
[]
[4]
['dog',4]
[True,'dog',4]
[True,'dog',4]
[True,'dog',4]
[8.4,True,'dog',4]
[8.4,True,'dog']
[8.4,True]
[8.4,True]
```

Enqueue:
(rear of the queue)
(beginning of the list)

True

3

False

4

'dog'

2



Exercise

Given the following implementation of the `enqueue()` and `dequeue()` methods, what are their Big-O complexities?

```
def enqueue(self, item):  
    self.items.insert(0, item)
```

```
def dequeue(self):  
    return self.items.pop()
```

- (a) The complexity of both methods is $O(1)$.
- (b) The complexity of both methods is $O(n)$.
- (c) The complexity of the `enqueue()` method is $O(1)$ and the `dequeue()` method $O(n)$.
- (d) The complexity of the `enqueue()` method is $O(n)$ and the `dequeue()` method $O(1)$.
- (e) None of the above.



Exercise

Given that the Stack and Queue classes have been implemented correctly, what is the output of the following code:

```
s1 = Stack()
q = Queue()
s1.push(1)
s1.push(2)
s1.push(3)
while not s1.is_empty():
    q.enqueue(s1.pop())
print(q.dequeue(), end = ' ')
print(q.dequeue(), end = ' ')
print(q.dequeue(), end = ' ')
```

- (a) 2 1 3
- (b) 3 2 1
- (c) 3 1 2
- (d) 1 2 3
- (e) 1 3 2

Circular Queue

```
def enqueue(self, item):  
    self.items.insert(0,item)  
def dequeue(self):  
    return self.items.pop()
```

- ▶ What is the Big-O performance of enqueue and dequeue of the implementation using Python List?

- ▶ enqueue(...): **$O(n)$**

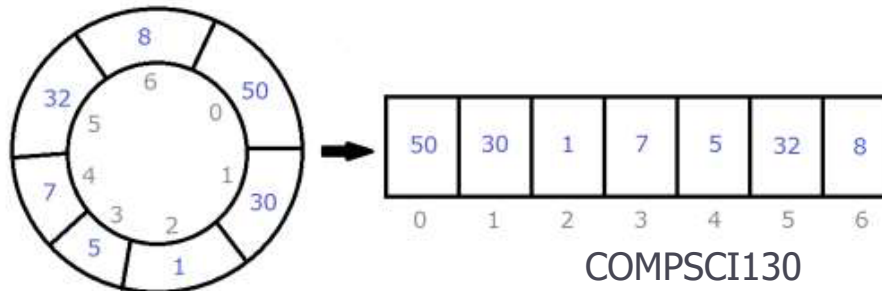
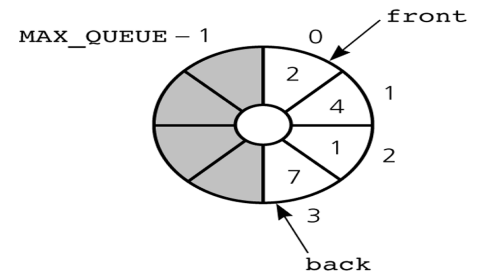
- ▶ Shifting array elements to the right after each addition – too Expensive!

- ▶ dequeue() : **$O(1)$**

- ▶ Another Implementation: **Circular Queue**

- ▶ enqueue & dequeue : **$O(1)$**

- ▶ Items can be added/removed without shifting the other items in the process

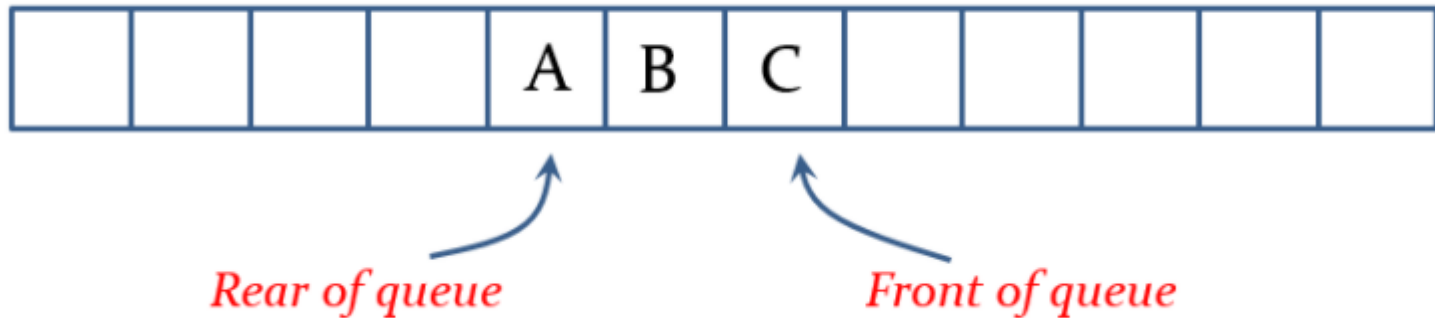


Viewed as a circle
instead of line



Circular Queue - Visualization

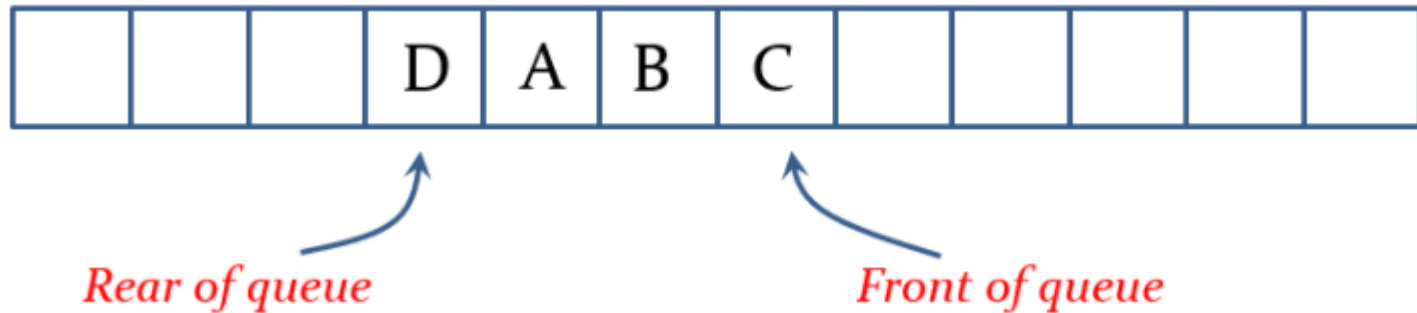
Imagine we had our queue elements stored somewhere in the middle of a long list with the front and end of the queue as shown below:



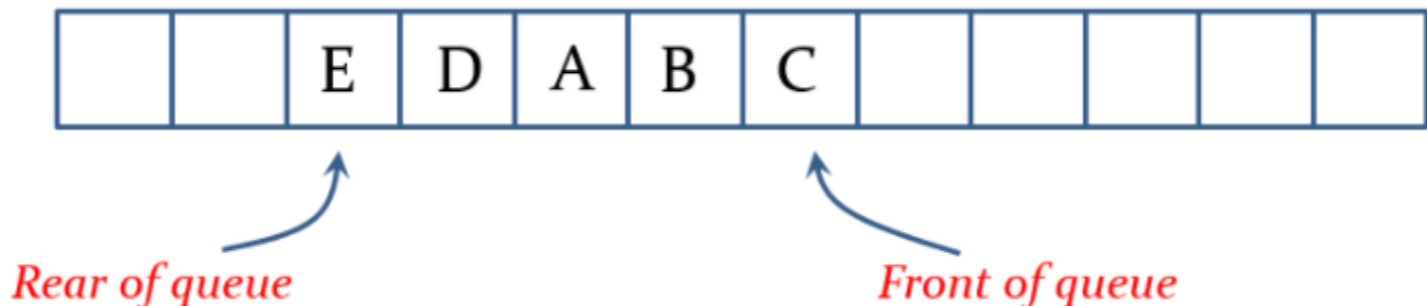


Circular Queue – Visualizing Enqueuing

Adding “D” to our queue would result in it looking like this:



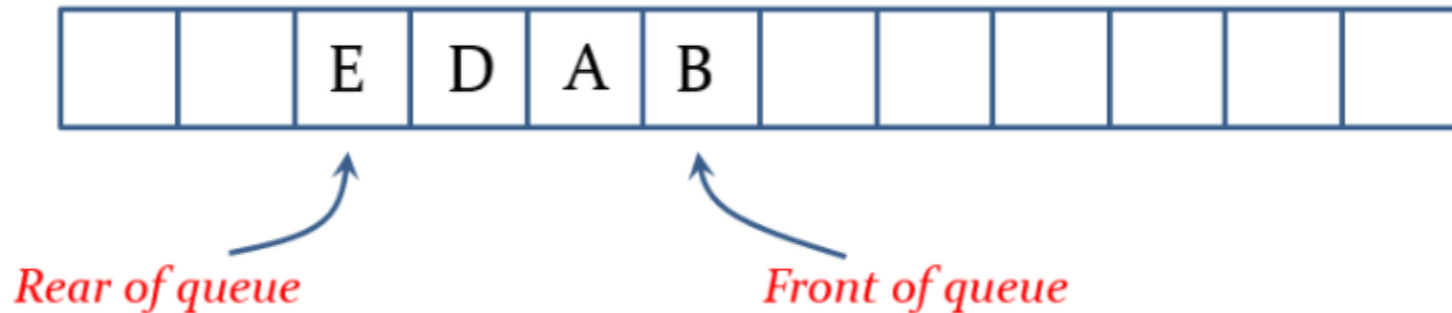
Notice that we did not need to shift any existing elements in the queue. Similarly we could add “E” to the queue without shifting anything:



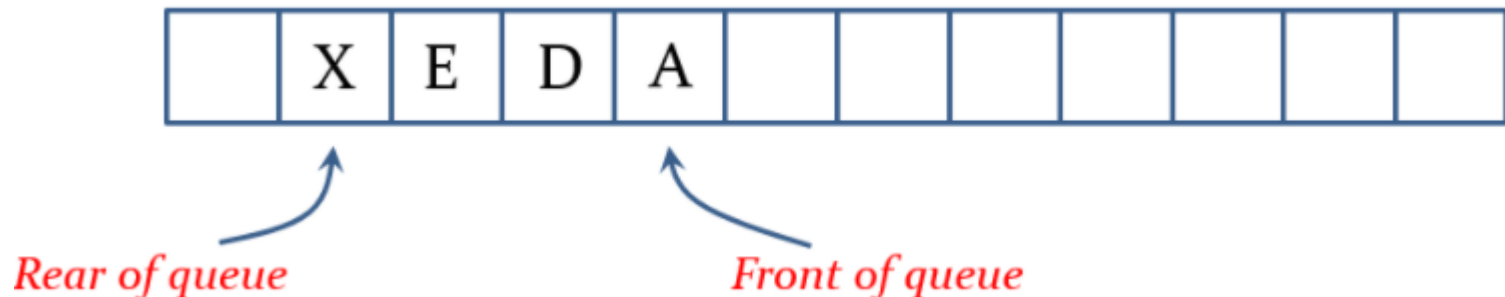


Circular Queue – Visualizing Dequeueing

Now let's remove the first item in the queue:

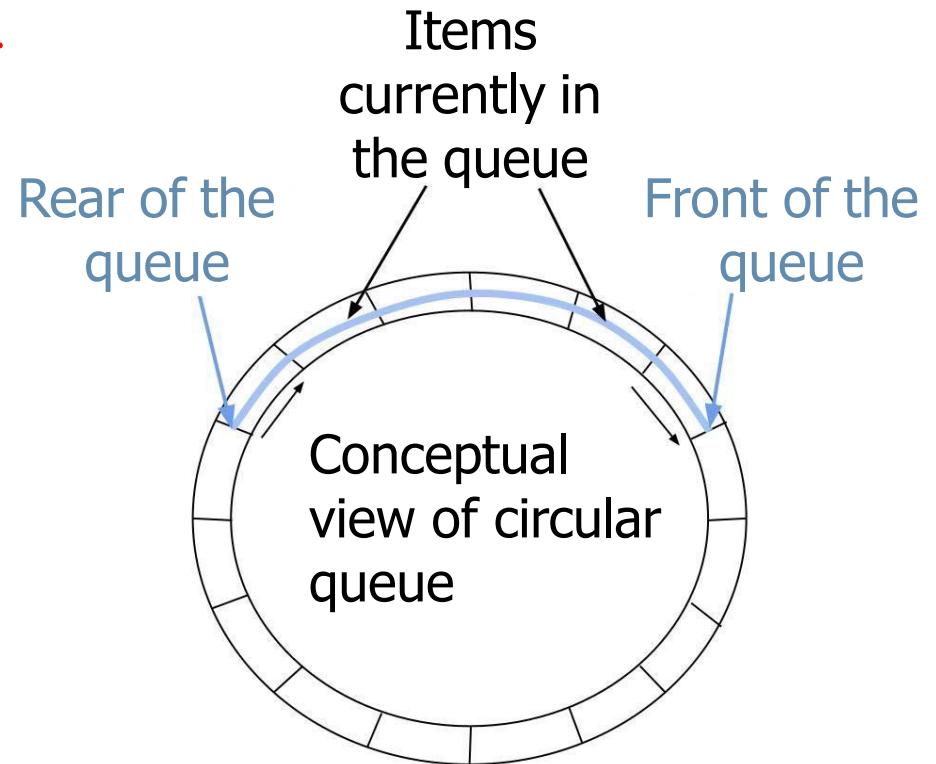


We did not need to shift any items in the queue when dequeueing. Let's remove the first item in the queue followed by enqueueing "X":



Circular Queue – Implementation Setup

- ▶ If we have a **fixed-sized list**, we can imagine it (conceptually) as a *circle* where the end of the list (at index position $\text{size} - 1$) wraps back around to the start of the list (at index position 0)
 - ▶ Items can be added and removed without shifting the other items in the process.
 - ▶ Enqueue and dequeue: Big-O of $O(1)$.





Circular Queue – Implementation Setup

- ▶ Uses a Python **list** data structure to store the items in the queue.
- ▶ The list has an initial capacity (all elements None)
- ▶ Keeps an index of the current **front** of the queue and of the current **back** of the queue.
 - ▶ set **front** to 0,
 - ▶ set **back** to $\text{max_queue} - 1$,
 - ▶ set **count** to 0
- ▶ New items are **enqueued** at the **back** index position
- ▶ Items are **dequeued** at the **front** index position.
- ▶ A **count** of the queue items to detect queue-full and queue-empty conditions

To initialize the queue



Circular Queue – Implementation

- ▶ Use a Python list to store the items in the queue.
- ▶ The list has an initial fixed capacity (specified via the constructor)
- ▶ Initially, all elements will have the special value “None” (used to represent the absence of a value in Python)
- ▶ We will maintain the **index position** of the current front of the queue and of the current rear of the queue
- ▶ We will also record the maximum capacity of the queue (we cannot store more elements than this size) and the number of elements in the queue

```
def __init__(self, capacity):  
    self.items = [None] * capacity  
    self.max_queue = capacity  
    self.front = 0  
    self.back = max_queue - 1  
    self.count = 0
```



Circular Queue – Implementation

- ▶ Implementation of enqueue() and dequeue():
 - ▶ New items are enqueued at the **back** index position (but only if the queue is not already full):

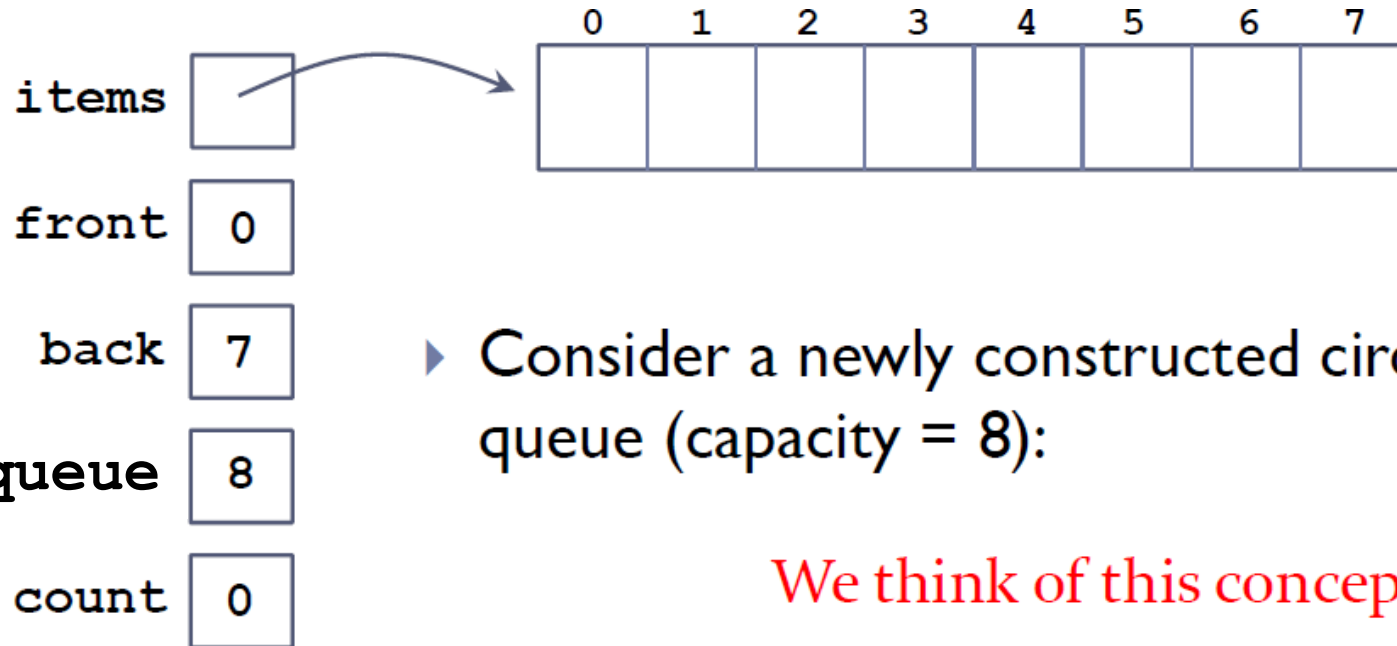
```
def enqueue(self, item):  
    if not self.is_full():  
        self.back = (self.back + 1) % self.max_queue  
        self.items[self.back] = item  
        self.count += 1
```

- ▶ Items are dequeued from the **front** index position (but only if the queue is not already empty):

```
def dequeue(self):  
    if not self.is_empty():  
        item = self.items[self.front]  
        self.front = (self.front + 1) % self.max_queue  
        self.count -= 1  
        return item
```

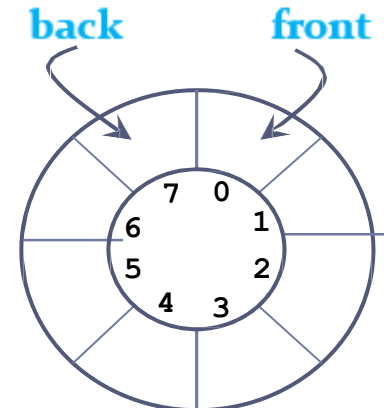
When either front or back advances past `max_queue - 1`, it wraps around to 0 using the modulus operator

Circular Queue – Implementation



- ▶ Consider a newly constructed circular queue (capacity = 8):

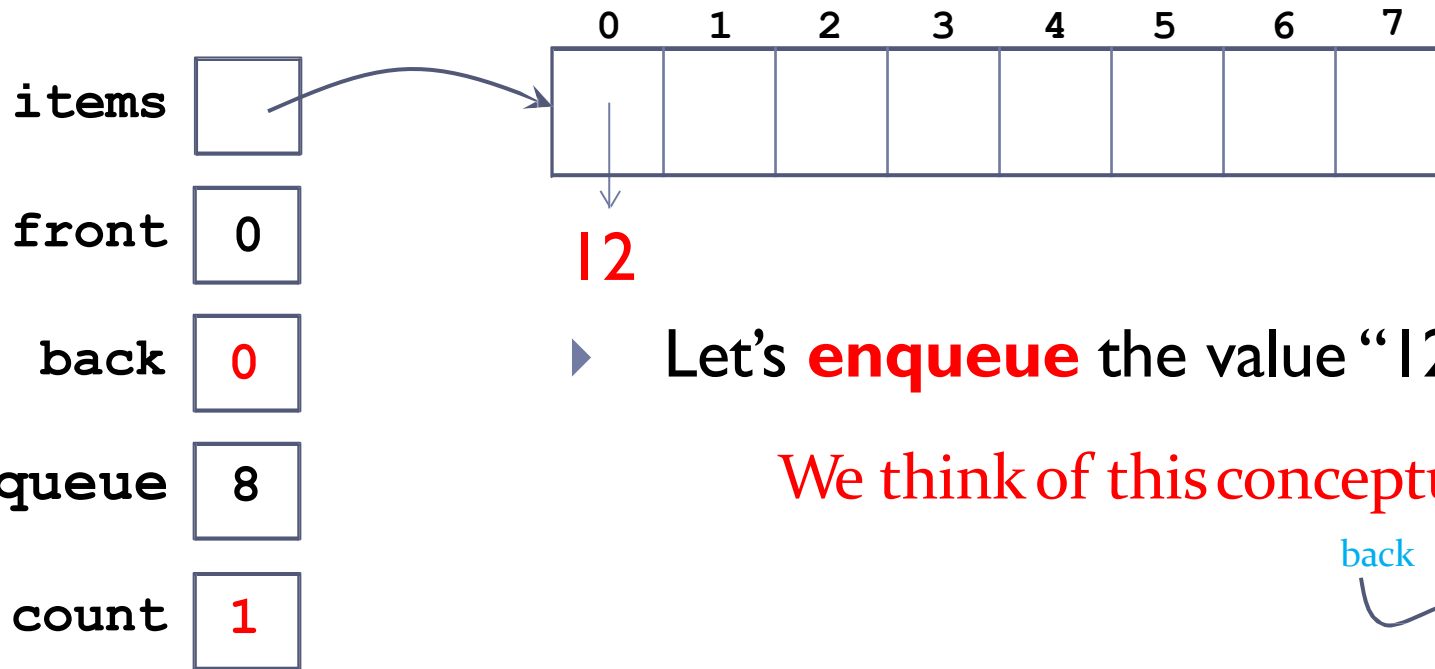
We think of this conceptually as:



```
def enqueue(self, item):  
    if not self.is_full():  
        self.back = (self.back + 1) % self.max_queue  
        self.items[self.back] = item  
        self.count += 1
```

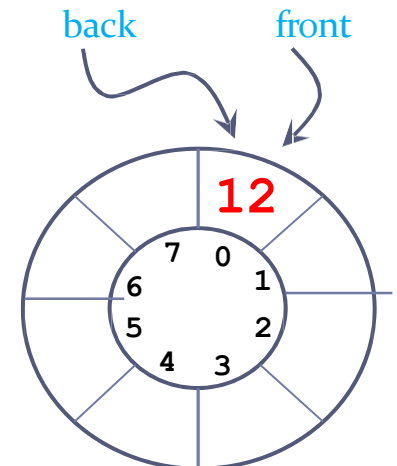


Circular Queue – Implementation



► Let's **enqueue** the value “12”:

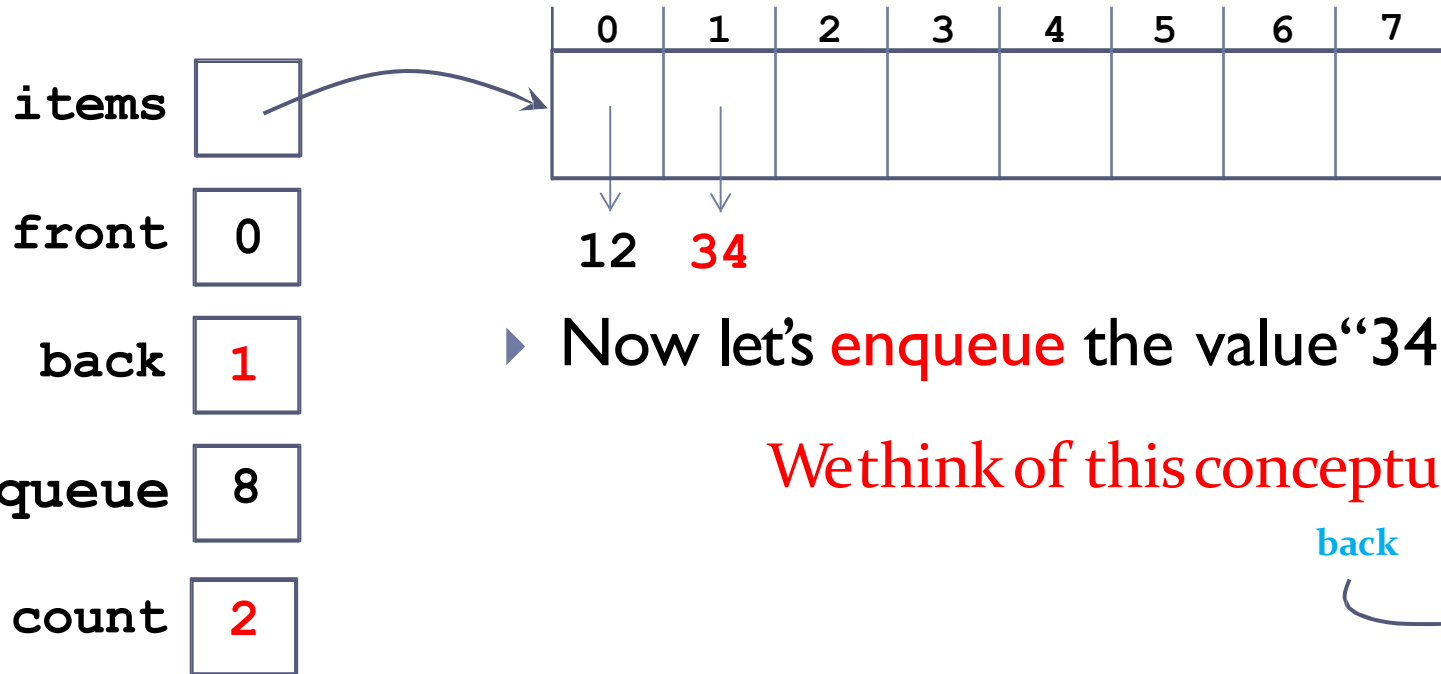
We think of this conceptually as:



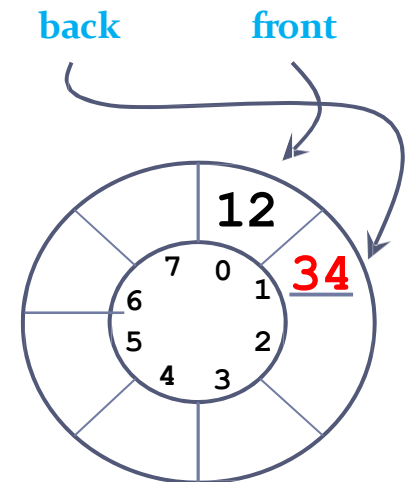
```
def enqueue(self, item):  
    if not self.is_full():  
        self.back = (self.back + 1) % self.max_queue  
        self.items[self.back] = item  
        self.count += 1
```



2. Circular Queue Implementation



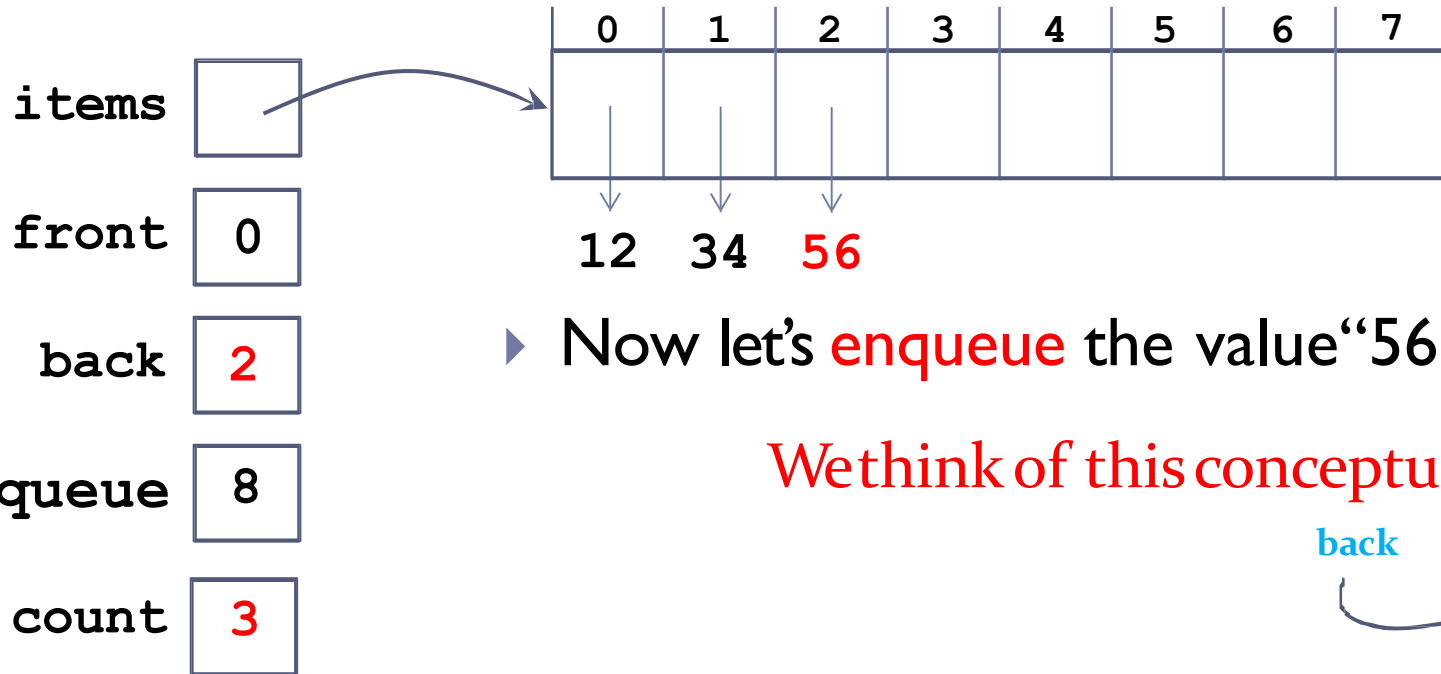
We think of this conceptually as:



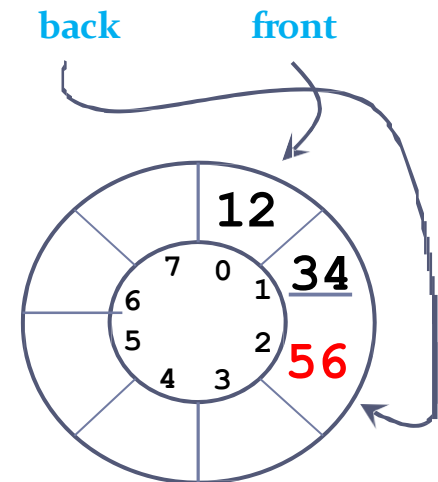
```
def enqueue(self, item):  
    if not self.is_full():  
        self.back = (self.back + 1) % self.max_queue  
        self.items[self.back] = item  
        self.count += 1
```



2. Circular Queue Implementation



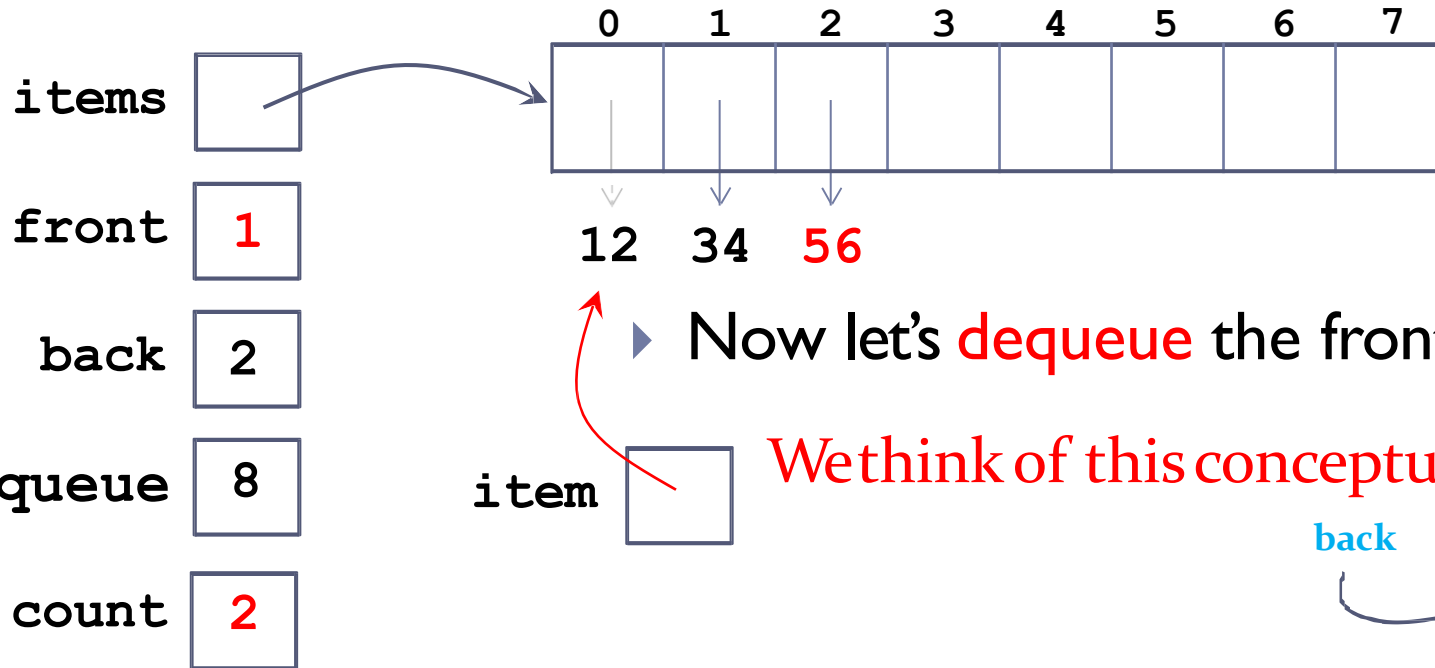
We think of this conceptually as:



```
def enqueue(self, item):  
    if not self.is_full():  
        self.back = (self.back + 1) % self.max_queue  
        self.items[self.back] = item  
        self.count += 1
```

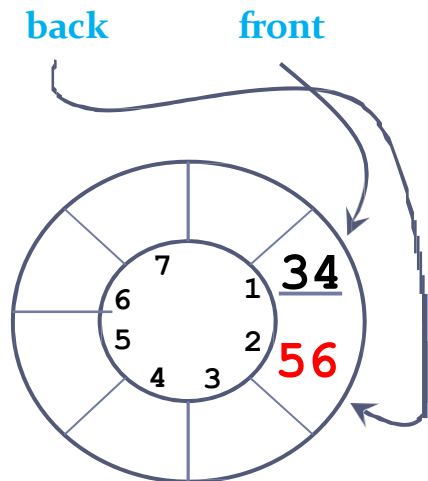


2. Circular Queue Implementation



Now let's **dequeue** the front value:

We think of this conceptually as:

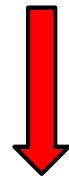
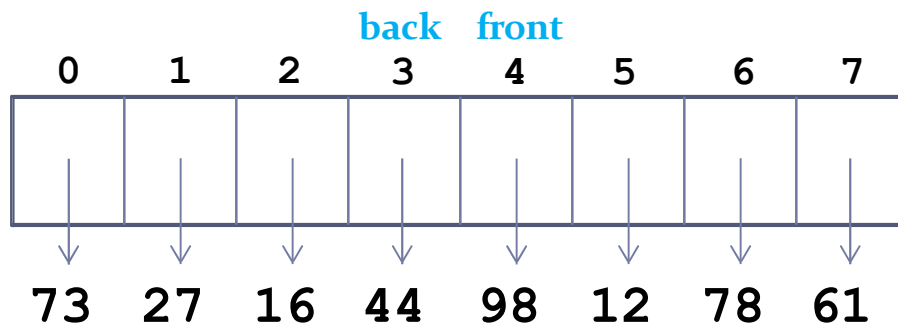


```
def dequeue(self):  
    if not self.is_empty():  
        item = self.items[self.front]  
        self.front = (self.front + 1) % self.max_queue  
        self.count -= 1  
        return item
```

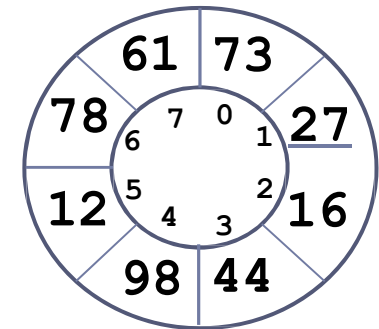
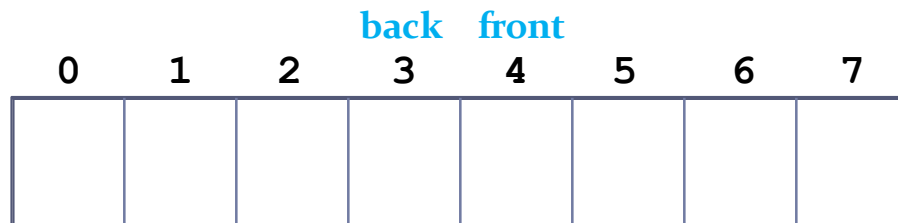


2. Circular Queue Implementation

- ▶ The values of front and back cannot be used to distinguish between a full queue and an empty queue.

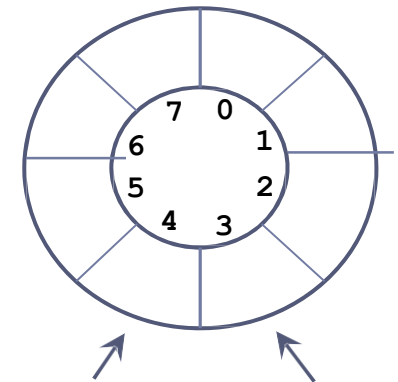


Dequeue all items



front

back



front

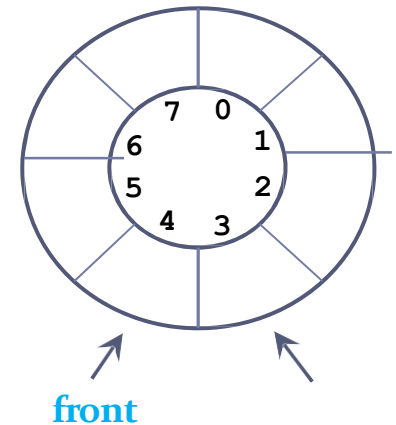
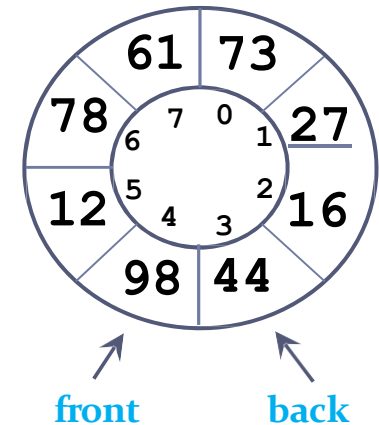


2. Circular Queue Implementation

- ▶ The values of front and back cannot be used to distinguish between a full queue and an empty queue.

```
def is_full(self):  
    return self.count == self.max_queue
```

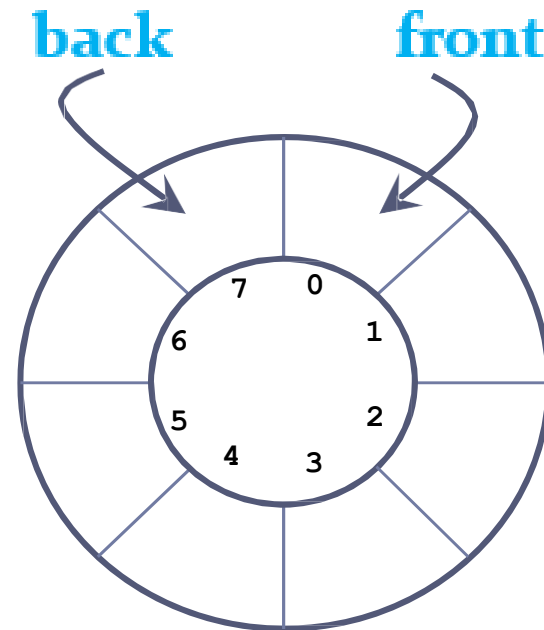
```
def is_empty(self):  
    return self.count == 0
```



Exercise

- ▶ What would the circular queue look like after the following code is executed?

```
q = Queue_Circular(8)
q.enqueue(5)
q.enqueue(6)
q.dequeue()
q.enqueue(7)
q.enqueue(8)
q.enqueue(9)
q.dequeue()
q.dequeue()
q.enqueue(10)
```





Exercise

- Which of the following operations in the Circular Queue implementation discussed in class has a complexity of $O(n)$?
- (a) `peek()`
 - (b) `is_empty()`
 - (c) `dequeue()`
 - (d) `enqueue()`
 - (e) None of the above.